Parallel Matrix Multiplication and Vectorisation

Javier Ruano Hernández
Big Data – Assignment 3
Universidad de Las Palmas de Gran Canaria (ULPGC)
Code and data: https://github.com/javierruanohdez/big-data-assignments/tree/main/assignment_3

***Abstract*—We parallelise the cache-blocked (tiled) dense matrix multiplication of Assignment 2 with OpenMP and add explicit SIMD vectorisation, in C. Three kernels with identical arithmetic and bit-identical results, a sequential one, a multithreaded one with a scalar inner loop and a multithreaded one with `#pragma omp simd`, were run for $n = 500$, 1000 and 2000 with $p = 1$, 2, 4 and 8 threads on a laptop CPU with 4 cores and 8 hardware threads, five times per configuration. Vectorisation was switched on and off loop by loop and verified from the compiler's report. At $n = 2000$, eight threads made the scalar kernel 4.56 times faster than the sequential kernel (efficiency 57%), and threads plus SIMD 10.9 times (0.445 s), the product of 4.56 from threads and 2.38 from SIMD. SIMD alone gave 3.75 with one thread, but its gain shrank with more threads: relative to itself, the vector kernel reached 2.53 on eight threads and the scalar kernel 3.99. Moving from 4 to 8 threads, onto SMT siblings, gave 1.45 for the scalar and 1.06 for the vector kernel. Estimated memory traffic rules out a DRAM bandwidth limit; the flattening is consistent with resources shared within and between cores. With one thread, the OpenMP kernel was 13–37% *faster* than the sequential one, so OpenMP's own overhead stayed below what this experiment resolves. These results describe one machine and one configuration.**

***Index Terms*—matrix multiplication, OpenMP, shared-memory parallelism, SIMD, vectorisation, speedup, parallel efficiency, simultaneous multithreading.**

I. Introduction and Objective

Assignment 2 made the dense matrix multiplication cache-friendly: loop reordering and tiling cut the time at $n = 2000$ from 15.6 to about 3.5 s. It left two resources unused. The kernels ran on one of the four cores of the test machine, and their inner loop processed one double per instruction; the report concluded that this scalar loop, not memory traffic, limited the tiled kernel. This report attacks both limits on the same tiled kernel: shared-memory multithreading with OpenMP [1], [2] distributes the work over the cores, and SIMD vectorisation processes several doubles per instruction [3].

a) Objective: We answer the following research question:

How does the time of the tiled matrix multiplication scale with the number of threads, why does it depart from the ideal, and how much does explicit SIMD vectorisation add on top of multithreading?

RQ1. What does the OpenMP machinery cost with a single thread ($p = 1$)?

RQ2. How do speedup and efficiency evolve with p and n, and where and why does scaling flatten?

RQ3. What happens when the threads exceed the physical cores and share them through simultaneous multithreading (SMT)?

RQ4. How much does SIMD add, and how do its gain and the threading gain combine?

b) Contributions and scope: We provide (i) three kernels in one program that differ in exactly one aspect each, with bit-identical results; (ii) a build that forces the inner loop to be scalar or vector, verified from the compiler's vectorisation report; (iii) an automated pipeline that validates, measures and analyses all configurations; and (iv) a measurement-based analysis that separates the threading and SIMD gains. Code and raw data are in the repository linked on the first page. The results describe one machine (Section III-A).

II. Background and References

A. Shared-Memory Parallelism with OpenMP

In OpenMP's fork-join model, a program runs on one thread until it reaches a parallel region, where a team of p threads starts; the threads share one address space [1], [2]. A `parallel for` construct divides the iterations of a loop among the team and ends with an implicit barrier, so the region lasts as long as its slowest thread. With `schedule(static)` the iterations are split into equal contiguous chunks before the loop starts; dynamic schedules hand out chunks at run time through a shared counter. Parallel execution has three costs that a sequential program does not pay: creating the threads (once per program in typical runtimes), forking and joining the team and synchronising at barriers, which microbenchmarks measure in microseconds [4], and load imbalance, since equal chunks finish at different times on unequal cores.

B. False Sharing

Caches keep cores coherent at the granularity of a cache line, 64 bytes on current x86-64 CPUs [3], [5]. When two cores write different variables that lie in the same line, each write invalidates the other core's copy, and the line bounces between their caches as if the data were shared [6]. A classic trap is an array of per-thread counters updated in an inner loop. It is avoided by giving every thread a disjoint region of output whose boundaries fall on cache-line boundaries, and by keeping accumulators private.

C. Speedup, Efficiency and Amdahl's Law

Let T_{seq} be the time of the sequential program and T_p the time with p threads. Speedup and efficiency are

$$S(p) = \frac{T_{\text{seq}}}{T_p}, \qquad E(p) = \frac{S(p)}{p}, \tag{1}$$

with ideal values $S(p) = p$ and $E(p) = 1$. If a fraction f of the work is serial, Amdahl's law bounds the speedup [7]:

$$S(p) \le \frac{1}{f + (1-f)/p}. \tag{2}$$

Gustafson observed that larger problems usually have a smaller serial fraction, so they scale better [8]. Inverting (2) for a measured speedup gives the Karp–Flatt metric [9],

$$e = \frac{1/S(p) - 1/p}{1 - 1/p}, \tag{3}$$

the serial fraction that would explain the measurement. It absorbs every loss, not only serial code: overheads, imbalance and contention for shared hardware. Finally, the roofline model [10] relates performance to the arithmetic intensity of a kernel (operations per byte moved from memory). A $b \times b$ tile product performs $2b^3$ floating-point operations on two input tiles of $8b^2$ bytes each, an intensity of $b/8$, i.e. 8 operations per byte for $b = 64$: tiling turns matrix multiplication from a memory-bound into a compute-bound kernel.

D. SIMD Vectorisation

SIMD instructions apply one operation to all lanes of a vector register: 4 doubles in a 256-bit AVX2 register, 8 in a 512-bit AVX-512 register [3]. Register width and hardware throughput may differ: the Zen 5 and Zen 5c cores of AMD's Strix Point mobile processors have 256-bit floating-point datapaths, half the width of an AVX-512 register [11]. The i-k-j inner loop $c_{ij} \mathrel{+}= a_{ik} b_{kj}$ over j suits SIMD perfectly: its iterations are independent and access B and C with unit stride. Compilers vectorise such loops automatically if they can prove that the iterations are independent; when two pointers may overlap, they add a run-time overlap check or keep the loop scalar. The OpenMP directive `#pragma omp simd` asserts independence and requests a vector loop [1]. A vector loop is followed by a short scalar or narrower *epilogue* for the remaining iterations. Because the lanes hold different output elements, vectorising over j does not change the order in which each c_{ij} accumulates its terms, unlike vectorising a sum.

E. Simultaneous Multithreading and Oversubscription

With SMT (Intel's Hyper-Threading), each core runs two hardware threads that share its execution units, load/store units and private caches. SMT raises throughput when one thread alone leaves resources idle, for example while waiting for memory, and helps little when a thread already saturates a shared resource [12], [3]. *Oversubscription* means more software threads than hardware threads: the operating system must time-slice them.

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT

Item	Value
CPU model	AMD Ryzen AI 5 330 w/ Radeon 820M
Cores / logical threads	4 cores (1 Zen 5 + 3 Zen 5c) / 8 threads (2-way SMT)
Clock	2.0 GHz base; boost up to 4.5 GHz (Zen 5 core) and 3.4 GHz (Zen 5c cores)
Caches	L1d 192 KiB, L1i 128 KiB, L2 4 MiB (i.e. 48 KiB, 32 KiB and 1 MiB per core); L3 8 MiB shared
SIMD	AVX2 and AVX-512; GCC used 64-byte (512-bit) vectors
RAM	15 GiB (available to WSL 2)
Operating system	Ubuntu 26.04 LTS on WSL2 (Windows Subsystem for Linux)
Kernel	6.18.33.2-microsoft-standard-WSL2
C compiler / OpenMP	GCC 15.2.0 with its OpenMP runtime (libgomp)
Python	CPython 3.14.4
Analysis	pandas 3.0.3, Matplotlib 3.10.9

III. Methodology

A. Experimental Environment

All measurements were taken on the machine described in Table I, with other applications closed and the computer connected to mains power. Its four cores are of two types, one Zen 5 core and three Zen 5c cores with lower maximum clocks [13].

B. Implementations

One C file, `parallel_mult.c`, contains three kernels; all use 64-bit doubles, flat row-major arrays, and the input loader and timer of the previous assignments.

a) `seq_tiled`: The tiled kernel of Assignment 2 ($b = 64$; tile loops *ii-jj-kk* with i-k-j inside), unchanged except for one directive on its innermost loop (see below). It is the sequential baseline.

b) `omp_tiled`: The same loops, with `#pragma omp parallel for schedule(static)` on the outermost tile loop (Listing 1). One iteration of *ii* computes a whole block-row of C (64 rows, all columns, all kk), so different threads never write the same element: no data races, no locks, atomics or reductions, and every c_{ij} still receives its terms in increasing k. Parallelising the kk loop instead would make threads add into the same tile of C concurrently. The static schedule gives each thread a contiguous chunk of row tiles at no run-time cost; since full tiles cost the same, it is balanced up to the partial last tile: the largest share exceeds the mean by 2.4% for every measured n and $p > 1$. `default(none)` forces every shared variable to be declared, so none can be shared by accident. Each thread clears its own block-row of C immediately before accumulating into it, rather than clearing all of C serially, which would add a serial part to the timed region.

c) False sharing: The only shared data written is C, and there are no shared counters or accumulators. Thread boundaries lie at rows that are multiples of b, at byte offset $8bnr$ for an integer r. C is allocated 64-byte aligned

```
#pragma omp parallel for schedule(static) \
        default(none) shared(a, b, c, n, bs)
for (ii = 0; ii < n; ii += bs) {
 memset(&c[ii*n], 0, rows(ii)*n*8); /* own rows */
 for (jj = 0; jj < n; jj += bs)
  for (kk = 0; kk < n; kk += bs)
   for (i = ii; i < min(ii+bs, n); i++)
    for (k = kk; k < min(kk+bs, n); k++) {
     a_ik = a[i*n + k];
     #pragma omp simd  /* omp_tiled: novector */
     for (j = jj; j < min(jj+bs, n); j++)
      c[i*n + j] += a_ik * b[k*n + j];
    }
}
```

Listing 1. `omp_simd_tiled` (simplified). `omp_tiled` is identical except for the inner-loop directive; `seq_tiled` has neither the outer pragma nor the per-block clearing.

(`aligned_alloc`), so with $b = 64$ every boundary is a cache-line boundary and no line is ever written by two threads, for any n.

d) omp_simd_tiled: `omp_tiled` with one change: `#pragma omp simd` instead of `#pragma GCC novector` on the innermost loop (Listing 1). The assertion of independence holds, since B and C are separate arrays and iteration j updates only c_{ij}; it lets GCC emit the vector loop without a run-time overlap check.

e) Forcing scalar and vector loops: All kernels are compiled with `-O3 -march=native`, and at `-O3` GCC auto-vectorises the i-k-j inner loop of *every* kernel. The comparison of `omp_tiled` with `omp_simd_tiled` would then compare vector code with vector code. The two reference kernels therefore mark their inner loop `#pragma GCC novector` (GCC 14 and later), which keeps that loop scalar while `-O3` still optimises it in every other way; the program refuses to compile with older versions, which would ignore the directive. In addition, `-std=c11` implies `-ffp-contract=off`: no multiply and add is fused into one FMA instruction, so the scalar and vector kernels round identically.

f) Thread control: The program disables dynamic adjustment of the team size with `omp_set_dynamic(0)` and sets it from its `-threads` argument with `omp_set_num_threads`, which overrides `OMP_NUM_THREADS`. It checks before timing that the team really has p threads; it prints the team size, and the harness records any mismatch as an error. Most OpenMP runtimes, including libgomp, create their threads at the first parallel region. In our program this is an untimed region that fills C with NaN using the same p; it also maps every page of C before the timer starts and exposes any element a kernel fails to write. The timed kernel thus reuses a running team, and the comparison of `omp_tiled` at $p = 1$ with `seq_tiled` measures the per-call cost of OpenMP (fork, join, schedule computation), not thread creation.

C. Inputs and Experimental Design

Inputs are the shared binary files of the previous assignments (`common/data_generator.py`): n, followed by A and B as little-endian doubles drawn uniformly from $[-1, 1)$ with a seed derived from 42 and n. Table II lists the parameters. At $n = 500$ the three matrices (5.7 MB) fit in the 8 MB L3 cache; at $n = 1000$ and 2000 (22.9 and 91.6 MB) they do not. The thread counts double from 1 to the 8 logical CPUs; $p = 16$ was configured but skipped by the harness, which never runs more threads than logical CPUs. With `OMP_PLACES=cores` and `OMP_PROC_BIND=spread`, threads are pinned to cores and spread over them: for $p \le 4$ each thread has its own core, and $p = 8$ places two threads on every core. `seq_tiled` runs only with $p = 1$.

TABLE II
FIXED EXPERIMENTAL PARAMETERS

Parameter	Value
Sizes	n = 500, 1000, 2000 (dense)
Kernels	`seq_tiled` ($p = 1$), `omp_tiled`, `omp_simd_tiled`; tile edge $b = 64$
Threads	p = 1, 2, 4, 8 ($p = 16$ skipped: more than the 8 logical CPUs)
Repetitions	5 per configuration, each in a fresh process; 300 s budget per run
Inputs	uniform in $[-1, 1)$, base seed 42, shared binary files
Compiler	`gcc -O3 -march=native -ftree-vectorize -fopenmp -std=c11 -Wall -Wextra` (all kernels)
OpenMP	`OMP_PLACES=cores`, `OMP_PROC_BIND=spread`; dynamic adjustment off
Timer	`clock_gettime(CLOCK_MONOTONIC)`, ns resolution
Memory	peak RSS (GNU `time -v`)

D. Measurement Protocol

The program times only the kernel call, with `CLOCK_MONOTONIC`; this is wall-clock time, which a speedup requires (CPU-time clocks add up the time of all threads). Input loading, allocation, the NaN pre-fill, thread creation and process start-up are excluded. A harness (`run_benchmark.py`) runs every repetition in a fresh process and records the peak resident set size (RSS) with GNU `time -v`. Runs exceeding the 300-s budget are recorded as timeouts. Since all kernels and thread counts produce bit-identical results, a run whose checksum differs from the first one recorded for its input file would be flagged as a mismatch and excluded.

E. Metrics

$S(p)$ and $E(p)$ follow (1), with $T_{\text{seq}}(n)$ the median time of `seq_tiled` and $T_p(n)$ the median time of the kernel with p threads at the same n. Because `seq_tiled` is scalar, $S(p)$ of `omp_simd_tiled` contains both the threading and the SIMD gain. Two further metrics separate them: the *self-relative speedup* $S_{\text{self}}(p) = T(1)/T(p)$ of each kernel against itself on one thread, and the *SIMD gain* $G(p) = T_{\text{omp}}(p)/T_{\text{simd}}(p)$ at equal p. They combine exactly as

$$S_{\text{simd}}(p) = \frac{T_{\text{seq}}}{T_{\text{simd}}(p)} = S_{\text{omp}}(p) \cdot G(p). \tag{4}$$

We report medians and interquartile ranges (IQR = $Q_3 - Q_1$) over the five repetitions [14], [15]; speedups are ratios of medians. The bands in Figs. 2 and 3 run from $T_{\text{seq}}/Q_3(T_p)$ to $T_{\text{seq}}/Q_1(T_p)$, the spread of the parallel runs. All statistics and figures are produced by `plot_results.py` from the raw data.

F. Validation

Before benchmarking, `verify_correctness.py` runs every kernel on six cases: $A \times I$, $A \times 0$, a hand-computed 3×3 product, and random inputs with n = 10, 100 and 131 (full and partial edge tiles; with n = 131 every tile edge is uneven). Each case runs with b = 64 and b = 7 (many tiles per thread, and inner loops shorter than and not a multiple of the vector width, so the epilogue runs), and the OpenMP kernels with 1, 2, 3, 4, 8 and 16 threads (uneven splits and, for small n, threads without work). Since IEEE 754 addition is not associative [16], each result is compared with a correctly rounded `math.fsum` reference using the tolerances of the previous assignments, derived from the dot-product error bound [17]: an element passes if

$$|x - x_{\text{ref}}| \le \tau_{\text{abs}} + \tau_{\text{rel}}\,|x_{\text{ref}}|, \tag{5}$$

with $\tau_{\text{rel}} = 10^{-12}$, $\tau_{\text{abs}} = 100\,n^2u$ and $u = 2^{-53}$. All 156 runs passed: the identity, zero and 3×3 cases were reproduced exactly, the largest error was 1.1×10^{-14} (random cases, n = 100 and 131), four orders of magnitude below τ_{abs}, and every result was bit-identical to that of `seq_tiled`. The script also parses GCC's vectorisation report (`-fopt-info-vec-optimized`) and maps each vectorised loop to its function. The report lists the inner loop of `omp_simd_tiled` as "loop vectorized using 64 byte vectors", with a 32-byte epilogue, and no vectorised loop in `seq_tiled` or `omp_tiled`.

IV. Results

All 135 runs (3 sizes, 5 repetitions, 9 kernel and thread combinations) completed within the time budget, with no timeouts, errors or checksum mismatches. Throughout, 1 MB = 2^{20} bytes.

A. Execution Time

At n = 2000, `seq_tiled` takes 4.84 s (Fig. 1, Table III). `omp_tiled` falls from 4.23 s with one thread to 2.27, 1.53 and 1.06 s with 2, 4 and 8 threads. `omp_simd_tiled` takes 1.13 s with one thread, less than `omp_tiled` with four, and falls to 0.865, 0.470 and 0.445 s. Per multiply-add, `seq_tiled` costs 0.55–0.60 ns at every size, and `omp_simd_tiled` on one thread 0.14 ns at n = 1000 and 2000. The fastest configuration, `omp_simd_tiled` with eight threads at n = 2000, reaches 35.9 GFLOP/s.

With one thread, `omp_tiled` was faster than `seq_tiled` at every size: by 36.8% at n = 500 (46.2 vs. 73.1 ms), 24.8% at n = 1000 (0.410 vs. 0.545 s) and 12.6% at n = 2000 (4.23 vs. 4.84 s). At n = 500 and 1000 the ranges of the five runs do not overlap (e.g. 43.9–51.4 against 71.5–80.6 ms at n = 500); at n = 2000 they do (3.18–4.62 against 4.16–5.10 s),

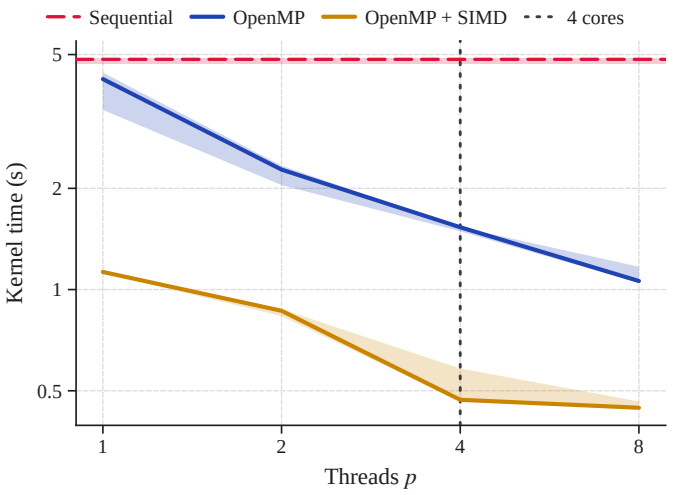


Fig. 1. Median kernel time over five runs vs. threads p at n = 2000 (log-log), with interquartile-range bands. Dashed horizontal line: `seq_tiled` with its IQR. Dotted vertical line: the 4 physical cores; p = 8 uses SMT.

TABLE III
Results at n = 2000: median and IQR of the kernel time over five runs, speedup and efficiency over `seq_tiled`, and self-relative speedup

		Time (s)				
Kernel	p	Median	IQR	$S(p)$	$E(p)$	$S_{\text{self}}(p)$
`seq_tiled`	1	4.839	0.18	1.00	100%	–
`omp_tiled`	1	4.227	1.00	1.14	114%	1.00
	2	2.274	0.29	2.13	106%	1.86
	4	1.532	0.043	3.16	79%	2.76
	8	1.060	0.12	4.56	57%	3.99
`omp_simd_tiled`	1	1.128	0.011	4.29	429%	1.00
	2	0.8648	0.041	5.59	280%	1.30
	4	0.4701	0.12	10.29	257%	2.40
	8	0.4451	0.020	10.87	136%	2.53

and the five `omp_tiled` runs with p = 1 form two groups, 3.18–3.42 s and 4.23–4.62 s. Most IQRs are below 15% of the median; the largest are 32% (`omp_tiled`, n = 1000, p = 2), 25% (`omp_simd_tiled`, n = 2000, p = 4) and 24% (`omp_tiled`, n = 2000, p = 1).

The peak RSS did not grow with p: it stayed within 93.2–93.9 MB at n = 2000, at most 3% above the 91.6 MB of the three matrices, and varied by less than 0.9 MB at every size. Threads add no measurable memory.

B. Speedup and Efficiency

At n = 2000, `omp_tiled` reaches $S(8)$ = 4.56 (Fig. 2): its efficiency falls from 106% at p = 2 to 79% at p = 4 and 57% at p = 8 (Fig. 3). Relative to itself, it scales by 1.86, 2.76 and 3.99. `omp_simd_tiled` lies above the ideal line at every p, with $S(8)$ = 10.87, but its curve flattens between p = 4 and 8 (S from 10.29 to 10.87), and relative to itself it reaches only 1.30, 2.40 and 2.53. Larger matrices scale better (Table IV): with eight threads, `omp_tiled` reaches $S(8)$ = 2.81, 3.88 and 4.56 at n = 500, 1000 and 2000 (efficiency 35,

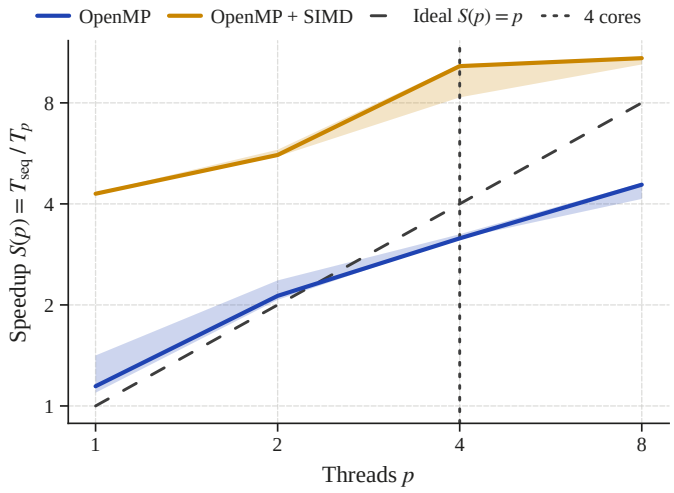


Fig. 2. Speedup $S(p) = T_{seq}/T_p$ vs. p at $n = 2000$ (log-log), with the ideal $S(p) = p$ (dashed). `omp_simd_tiled` includes the SIMD gain, since `seq_tiled` is scalar. Bands: $T_{seq}/Q_3(T_p)$ to $T_{seq}/Q_1(T_p)$.

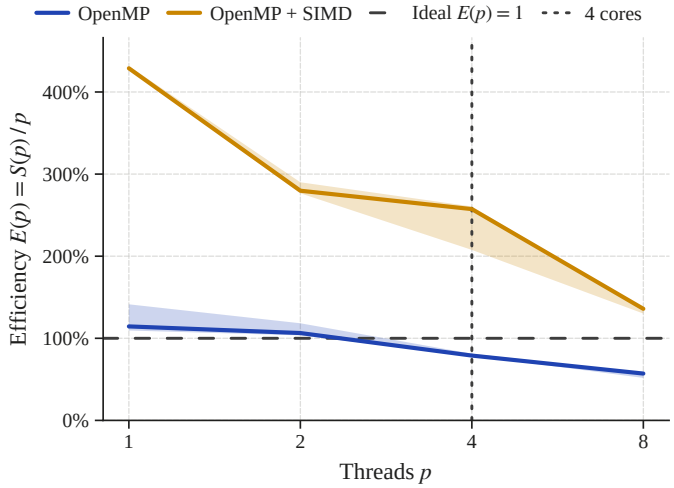


Fig. 3. Efficiency $E(p) = S(p)/p$ vs. p at $n = 2000$, with the ideal $E(p) = 1$ (dashed). Values above 100% reflect the SIMD gain (`omp_simd_tiled`) or the faster single-thread code of `omp_tiled` compared with `seq_tiled` (Section V-A), not superlinear threading.

48 and 57%), and $S_{self}(8) = 1.78$, 2.91 and 3.99. At $n = 500$, `omp_simd_tiled` is fastest with four threads (9.92 ms, $S = 7.37$) and slower with eight (12.6 ms).

C. SIMD Gain and SMT

Vectorisation sped up the parallel kernel by 1.84–3.75 with one thread and by 2.05–2.38 with eight (Table V). At $n = 2000$, (4) splits the total speedup of `omp_simd_tiled` on eight threads, 10.87, into 4.56 from threads and 2.38 from SIMD. Going from four threads (one per core) to eight (two per core), `omp_tiled` gained 1.22–1.45, whereas `omp_simd_tiled` gained 1.06 at $n = 2000$ and 1.30 at $n = 1000$, and at $n = 500$ eight threads took 27% longer than four (0.79).

TABLE IV
SPEEDUP $S(p)$ OVER `SEQ_TILED` AT EVERY SIZE, AND SELF-RELATIVE SPEEDUP WITH EIGHT THREADS. MEDIAN `SEQ_TILED` TIMES: 0.0731, 0.5453 AND 4.839 S FOR $n = 500$, 1000 AND 2000

n	Kernel	$S(1)$	$S(2)$	$S(4)$	$S(8)$	$S_{self}(8)$
500	`omp_tiled`	1.58	1.76	2.25	2.81	1.78
	`omp_simd_tiled`	2.91	4.81	7.37	5.78	1.99
1000	`omp_tiled`	1.33	1.71	3.18	3.88	2.91
	`omp_simd_tiled`	3.78	4.85	6.63	8.64	2.29
2000	`omp_tiled`	1.14	2.13	3.16	4.56	3.99
	`omp_simd_tiled`	4.29	5.59	10.29	10.87	2.53

TABLE V
SIMD GAIN $G(p) = T_{omp}(p)/T_{simd}(p)$ AND SMT GAIN $T(4)/T(8)$ OF THE SCALAR (`OMP_TILED`) AND VECTOR (`OMP_SIMD_TILED`) KERNELS (MEDIAN TIMES)

	SIMD gain $G(p)$				**SMT gain**	
n	$p = 1$	$p = 2$	$p = 4$	$p = 8$	Scalar	Vector
500	1.84	2.74	3.28	2.05	1.25	0.79
1000	2.84	2.83	2.09	2.23	1.22	1.30
2000	3.75	2.63	3.26	2.38	1.45	1.06

V. DISCUSSION

A. The Cost of OpenMP at One Thread

With $p = 1$, `omp_tiled` executes the loops of `seq_tiled` plus one parallel region with a team of one, a static schedule computation and a join: microseconds of work [4], less than 0.1% of even the 46-ms kernel at $n = 500$. OpenMP could make the kernel slightly slower, never 13–37% faster. The measured difference therefore does not measure framework overhead, which lies below the resolution of this experiment, but a difference between the two compiled kernels. They run the same source loops in different contexts: GCC compiles `seq_tiled` as a stand-alone function, whereas it moves the body of the parallel loop into a separate function that receives the matrix pointers, n and the block size through a structure and computes its own iteration range, so register allocation, address computation and unrolling may differ. Assignment 2 showed how much such context matters: inlining alone made the tiled kernel 2.5 times slower. We did not compare the machine code, so this remains a hypothesis. Two other explanations fail. The per-block clearing of C saves at most the cost of one 2-MB `memset` at $n = 500$, well under 1 ms of the 27 ms difference; and run order cannot explain it, since at $n = 500$ `seq_tiled` was the first configuration of the whole sweep, on a cool processor. At $n = 2000$, core placement adds to the effect (Section V-B). Two consequences follow: the efficiencies above 100% of `omp_tiled` at $p \le 2$ do not indicate superlinear threading, and the self-relative speedup is the more reliable measure of thread scaling here.

B. Scaling and Its Limits

a) Not Amdahl's serial code: The timed region contains no serial part beyond the fork and join: each thread clears its

own block-row and computes its own tiles, so f is of the order of 10^{-5} for a 1-s kernel and (2) allows a speedup of practically p. The measured scaling is far below that. The Karp–Flatt fraction (3) of `omp_tiled` at $n = 2000$, computed from its self-relative speedup, is 0.08, 0.15 and 0.14 for $p = 2$, 4 and 8. These values measure how much each thread slows down when others run, not serial code. Static scheduling explains only 2.4% (Section III-B).

b) Not a DRAM bandwidth wall: With 64×64 tiles, each block-row of C reads B once, so B ($8n^2 = 3.2 \cdot 10^7$ bytes) is read $n/b = 31.25$ times, $1.0 \cdot 10^9$ bytes, and all tile loads together are at most $2n^3/b$ doubles, $2.0 \cdot 10^9$ bytes. In the fastest run (0.445 s) this is at most 4.5 GB/s (10^9 bytes per second), a quarter of the 18.5 GB/s that a *single* core streamed in Assignment 2. At 8 operations per byte, the tiled kernels sit in the compute-bound region of the roofline model [10]: tiling removed the memory wall that limited the baseline of Assignment 2. The limits lie inside the processor.

c) Heterogeneous cores: A static, even split ends when the slowest thread ends. If the single-thread run used the Zen 5 core at up to 4.5 GHz and the four-thread run waited for Zen 5c cores at up to 3.4 GHz, a compute-bound kernel could reach at most $4 \times 3.4/4.5 \approx 3.0$ on four cores. The measured $S_{\text{self}}(4) = 2.76$ of `omp_tiled` lies just below this bound. Under WSL 2, thread pinning applies to virtual CPUs, and Windows decides which physical core runs each of them. The two groups of single-thread `omp_tiled` runs at $n = 2000$ have a ratio of 1.34, close to the clock ratio $4.5/3.4 = 1.32$: this is consistent with, but does not prove, the thread having run on different core types in different runs. Clocks also fall when more cores are active within a laptop's power budget; we could not measure frequencies under WSL 2.

d) Problem size: As Gustafson's argument suggests [8], larger problems scaled better: $S_{\text{self}}(8)$ of `omp_tiled` rose from 1.78 at $n = 500$ to 3.99 at $n = 2000$. The partial-tile imbalance is the same 2.4% at every size, and fork and join cost microseconds, so they do not explain the weak scaling at $n = 500$; with kernels of 10–46 ms and the largest relative spreads, we cannot attribute it to a single cause with the available data.

C. Beyond the Physical Cores: SMT and Oversubscription

With $p = 8$, two threads share each of the four cores. The ideal line of Fig. 2 still assumes $S(8) = 8$, i.e. eight independent cores; with four cores, it would require SMT to double the throughput of each. SMT raised the throughput of `omp_tiled` by 1.22–1.45 (Table V): one scalar thread leaves part of the core idle, and a sibling can use it [12]. For `omp_simd_tiled`, SMT gained only 1.06 at $n = 2000$, and at $n = 500$ eight threads took 27% longer than four. A vector thread already keeps the core's load/store units busy with two vector loads and one store per vector of results, and two siblings also share the 48 KiB L1 data cache. One thread's working set, a 32-KiB tile of B plus two row segments, fits in it; two such sets do not. Our interpretation is that the vector kernel saturates resources that SMT siblings share; hardware counters, which would confirm it, are not readily available under WSL 2. True oversubscription ($p = 16$, more threads than the 8 hardware threads) was not measured: the harness skips it by design, because its result would mainly reflect the operating system's time-slicing. With a static schedule, every time-sliced thread delays the final barrier, so no gain over $p = 8$ is expected.

D. Separating SIMD from Threading

Equation (4) separates the two effects. At $n = 2000$ with eight threads, threads contribute 4.56 and SIMD 2.38. With one thread, SIMD gave 2.84 at $n = 1000$ and 3.75 at $n = 2000$. A 64-byte vector holds 8 doubles, but if the cores of this processor, from the same mobile family, have the 256-bit datapaths of Strix Point [11], the hardware processes about 4 doubles per operation, for the arithmetic as well as for the loads and stores, so a gain near 4 is the realistic ceiling for this loop. $G(1)$ at $n = 2000$ is close to it, although it inherits the wide spread of the single-thread `omp_tiled` runs (IQR 24%). This supports the interpretation of Assignment 2 that the scalar inner loop was the bottleneck of the tiled kernel. The gains do not multiply, however. $G(p)$ shrinks with p at $n = 2000$ (3.75 at $p = 1$, 2.38 at $p = 8$) because `omp_simd_tiled` scales worse with threads than `omp_tiled` ($S_{\text{self}}(8) = 2.53$ vs. 3.99): the faster each thread computes, the more its speed depends on resources that threads share, SMT siblings within a core and the power budget across cores. For the vector kernel, a second thread per core paid off only at $n = 1000$ (1.30); at $n = 2000$, $p = 4$ was within 6% of $p = 8$, and at $n = 500$ it was faster. Overall, the fastest configuration (0.445 s at $n = 2000$) was 8.0 times faster than the best sequential kernel of Assignment 2 (3.55 s) and 35 times faster than the baseline of Assignment 1 measured there (15.6 s); these factors compare different sessions and compiler flags.

E. Limitations and Threats to Validity

- **Single machine, virtualised.** All results come from one laptop (Table I) under WSL 2, where thread pinning controls virtual, not physical, CPUs.
- **Heterogeneous cores.** One Zen 5 and three Zen 5c cores with different clocks make single-thread times depend on placement, which affects T_{seq} and every $S(p)$ (Section V-B).
- **Code generation.** The $p = 1$ comparison is confounded by how GCC compiled the two kernels (Section V-A); the machine code was not inspected, and no hardware counters or clock frequencies were measured.
- **Sessions and flags.** `seq_tiled` took 4.84 s at $n = 2000$, against 3.55 s for the same loops at `-O2` in Assignment 2. Flags, session and pinning differ; all comparisons here come from one session.
- **Scope.** One tile size ($b = 64$, not tuned for threads), one schedule (static), thread counts in powers of two, no FMA (`-ffp-contract=off`), five repetitions, and no measurement of true oversubscription ($p > 8$).

VI. Conclusions

We measured three tiled kernels in C with identical arithmetic and inputs: a sequential one, an OpenMP one with a scalar inner loop and an OpenMP one with explicit SIMD. On the tested machine (an AMD Ryzen AI 5 330 with 4 cores and 8 hardware threads, running Ubuntu 26.04 under WSL 2) with GCC 15.2.0 `-O3 -march=native` and $b = 64$, we conclude that:

1) **OpenMP's own cost was not measurable.** With one thread, the OpenMP kernel was 13–37% *faster* than the sequential one, most likely an effect of code generation (and, at $n = 2000$, core placement) rather than of the framework, whose fork and join cost microseconds.
2) **Threads scaled well but not ideally.** The scalar kernel was 4.56 times faster with eight threads at $n = 2000$ (efficiency 57%; 3.99 relative to itself) and scaled better for larger n. The losses are not due to serial code (none in the timed region) or DRAM bandwidth (at most 4.5 GB/s used); they are consistent with shared hardware and unequal cores.
3) **SMT helped the scalar kernel more than the vector kernel.** Eight threads on four cores gave 1.22–1.45 over four threads for the scalar kernel, but 0.79–1.30 for the vector kernel.
4) **SIMD gave up to 3.75 on one thread, near the ~4 that a 256-bit datapath allows, but less with more threads** (2.38 on eight at $n = 2000$). Combined, threads and SIMD made the kernel 10.9 times faster than the sequential one (4.56 from threads × 2.38 from SIMD), at 0.445 s.

These conclusions apply only to the hardware, software, settings and sizes described here; other processors (in particular with identical cores or 512-bit datapaths), compilers, tile sizes, schedules or operating systems could change both the absolute numbers and the scaling.

References and Materials

a) Materials: All code, raw data and scripts are in the public repository linked on the first page:

- `common/data_generator.py`: shared inputs;
- `assignment_3/src/c/parallel_mult.c`: the three kernels;
- `assignment_3/verify_correctness.py`: correctness and vectorisation-report checks;
- `assignment_3/run_benchmark.py`: benchmark harness;
- `assignment_3/plot_results.py`: statistics and figures;
- raw data (`measurements.csv`) and GCC's vectorisation report (`vectorization_report.txt`) in `assignment_3/results/`;
- `assignment_3/README.md`: exact commands to reproduce every result in this report.

[1] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, Nov. 2021.

[2] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA, USA: MIT Press, 2008.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] J. M. Bull, "Measuring synchronisation and scheduling overheads in OpenMP," in *Proc. 1st European Workshop on OpenMP (EWOMP)*, 1999, pp. 99–105.

[5] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., 2007.

[6] J. Torrellas, M. S. Lam, and J. L. Hennessy, "False sharing and spatial locality in multiprocessor caches," *IEEE Transactions on Computers*, vol. 43, no. 6, pp. 651–663, 1994.

[7] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[8] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[9] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Communications of the ACM*, vol. 33, no. 5, pp. 539–543, 1990.

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[11] TechPowerUp, "AMD Strix Point SoC 'Zen 5' and 'Zen 5c' CPU cores have 256-bit FPU datapaths," Jul. 2024. [Online]. Available: https://www.techpowerup.com/324873/

[12] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: Maximizing on-chip parallelism," in *Proc. 22nd Annu. Int. Symp. on Computer Architecture (ISCA)*, 1995, pp. 392–403.

[13] Notebookcheck, "AMD Ryzen AI 5 330 processor – benchmarks and specs." [Online]. Available: https://www.notebookcheck.net/AMD-Ryzen-AI-5-330-Processor-Benchmarks-and-Specs.1049553.0.html

[14] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. on Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.

[15] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. ACM SIGPLAN Int. Symp. on Memory Management (ISMM)*, 2013, pp. 63–74, doi: 10.1145/2464157.2464160.

[16] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019.

[17] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

AI Usage Disclosure

Generative AI tools were used to support the development of this assignment. Gemini (Google) was used for high-level project planning, repository structuring, and ensuring compliance with the assignment rubric. Claude (Anthropic) was used to draft the initial source code (C kernels and Python scripts), the automated benchmarking and plotting scripts, and the LaTeX report template.

All AI-generated code and text were thoroughly reviewed, tested, and adapted by the author. The author remains fully responsible for understanding the implementations, executing the experiments on local hardware, validating the measurements, and ensuring the mathematical and factual correctness of every statement in this report.